

**Part 1: Warm-up****1.1 Data visualization****Setup**

The three toy datasets (swiss roll, 9-mode gaussians, and concentric circles) are loaded at  $D = 2$  using the provided dataloader `ToyDiffusionDataset()` and visualised with matplotlib scatter plots. Each dataset is also loaded at  $D = 32$  and projected back to 2D using the `_2d()` method to verify that the projection preserves the original structure.

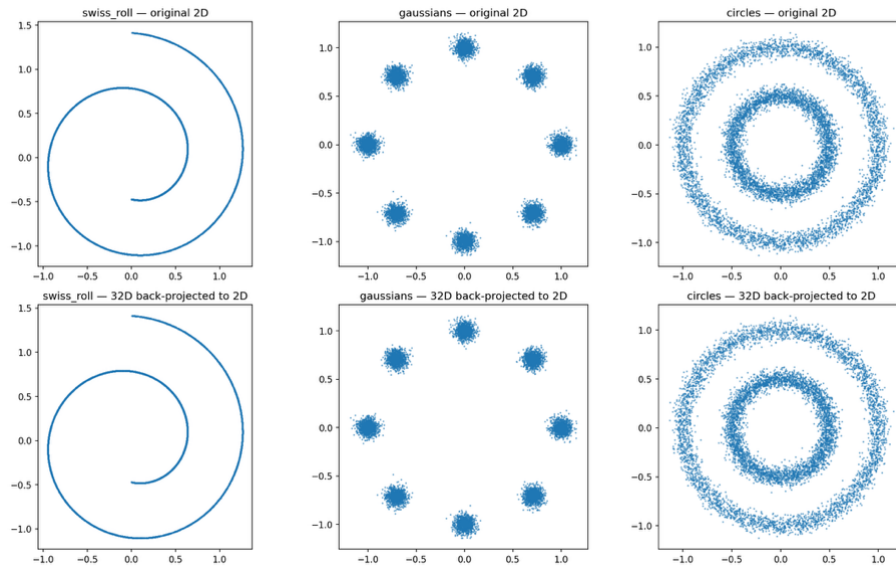
**Results**

Figure 1.1: Top row: original 2D datasets. Bottom row: 32D data projected back to 2D.  
Left to right: swiss roll, 8-mode gaussians, concentric circles.

The original 2D data shows the expected structure: a spiral for swiss roll, 8 distinct Gaussian clusters arranged in a ring for gaussians, and two concentric rings for circles. The 32D back-projected data is visually identical to the original 2D data, confirming that the random orthogonal projection used to embed the data into 32 dimensions is invertible and preserves the underlying geometry.

**1.2 v-Prediction Flow Matching at D=2****Implementation:**

The Flow Matching model is a 5-layer MLP with hidden width 256 and ReLU activations. It predicts a  $D$ -dimensional velocity vector using the concatenation of the noisy input  $z_t$  and a 128-dimensional sinusoidal timestep embedding. Therefore, the total input dimension is  $D + 128$ . **During training**, a clean data sample  $x$  is drawn from the dataloader. Gaussian noise  $\epsilon$  is sampled from a standard normal distribution, and a timestep  $t$  is sampled uniformly from

[0,1]. The noisy sample is constructed by linear interpolation between the clean data and noise:  $z_t = (1 - t)x + t\epsilon$ . The corresponding velocity target is  $v = \epsilon - x$ . The model is trained using v-prediction, meaning that it directly predicts the velocity field  $v_\theta(z_t, t)$ . The loss is computed as the MSE between the predicted velocity and the target velocity:  $L = ||v_\theta(z_t, t) - (\epsilon - x)||^2$ . All three datasets, swiss roll, gaussians, and concentric circles, are trained at  $D = 2$  using the Adam optimizer with a learning rate of 1e-3, batch size 1024, and 25,000 training steps. **For sampling**, generation starts from pure Gaussian noise at  $t = 1$  and solves the learned ODE backwards to  $t = 0$  (clean data) using 50 Euler steps. At each step, the sample is updated as  $z \leftarrow z - v \cdot dt$  where  $dt = \frac{1}{50}$ . For each dataset, 2048 samples are generated for visual comparison with the ground-truth distribution. The **hyperparameters** used are:

- **Prediction/loss:** v-prediction/v-loss
- **Architecture:** 5 hidden layers, 256 units, RELU
- **Time embedding:** 128-dim sinusoidal
- **Optimizer:** Adam,  $lr = 10^{-3}$
- **Batch size:** 1024
- **Training steps:** 25,000
- **Sampling:** Euler ODE, 50 steps
- **Num samples:** 2048

## Results

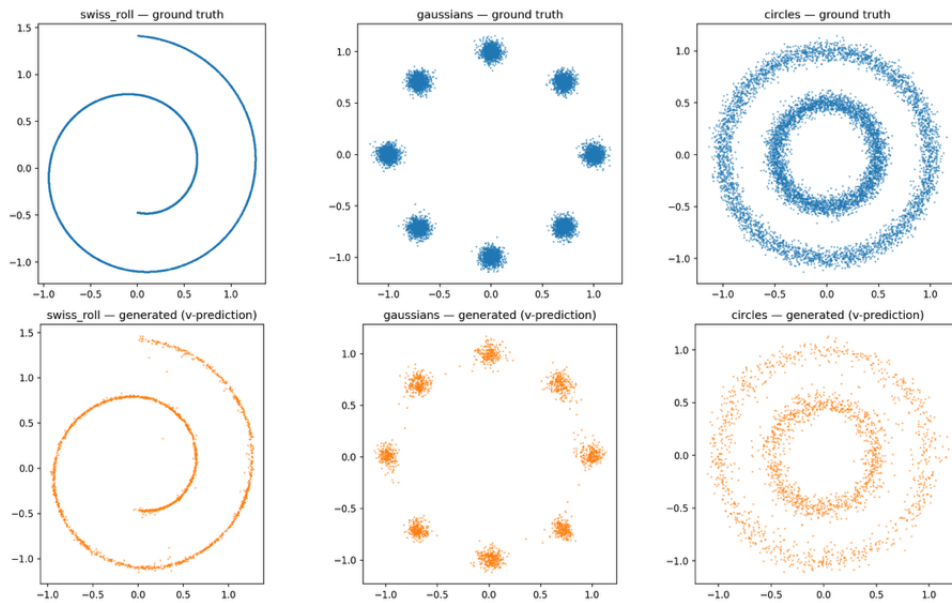


Figure 1.2: Top row: ground truth data. Bottom row: 2048 generated samples using v-prediction/v-loss at  $D=2$ . Left to right: swiss roll, 8-mode gaussians, concentric circles.

The generated samples closely match the ground truth across all three datasets. Swiss roll recovers the full spiral structure with correct density. Gaussians reproduces all 8 clusters at the correct locations with similar spread. Circles captures both concentric rings with clear separation. This confirms the implementation is correct before scaling to higher dimensions.

## Part 2: Flow Matching Parametrization

### 2.1 Calculate flow matching based on x or v-prediction

Given the forward process:  $z_t = (1 - t)x + t\epsilon$ , where  $v = \epsilon - x$ .

**From x-prediction to v:**

From the forward process, solve for  $\epsilon$ :

$$\epsilon = \frac{z_t - (1-t)x}{t}$$

Substitute into  $v = \epsilon - x$ :

$$v = \frac{z_t - (1-t)x}{t} - x = \frac{z_t - (1-t)x - tx}{t} = \frac{z_t - x + tx - tx}{t} = \frac{z_t - x}{t}$$

Therefore,  $v = \frac{z_t - \hat{x}}{t}$

**From v-prediction to x:**

From  $v = \epsilon - x$ , to get  $\epsilon = v + x$ .

Substitute into the forward process  $z_t = (1 - t)x + t\epsilon$ :

$$z_t = (1 - t)x + t(v + x) = (1 - t)x + tv + tx = x + tv$$

Rearrange:  $x = z_t - tv$

### 2.2 Reproduce the paper's finding

#### Implementation:

All four prediction/loss combinations are implemented: v-pred/v-loss, v-pred/x-loss, x-pred/v-loss, and x-pred/x-loss. Depending on the prediction type, the model directly outputs the velocity  $v$  or the clean data  $x$ . When the prediction space and loss space are different, the model output is converted before computing MSE loss. For v-pred/x-loss, the predicted clean data is computed as:  $x_{pred} = z_t - t \cdot v_{pred}$ ; For x-pred/v-loss, the predicted velocity is computed as:  $v_{pred} = \frac{(z_t - x_{pred})}{t}$ .

During sampling, Euler ODE integration is performed from  $t = 1$  to  $t = 0$ . The model output is converted into a velocity at each step. For v-prediction, the output can be used directly as the velocity. For x-prediction, the velocity is derived as  $v = \frac{(z_t - x_{pred})}{t}$ .

Initially, timesteps were clipped to  $[10^{-5}, 1 - 10^{-5}]$ . This caused numerical instability for settings involving division by  $t$  (x-pred/v-loss). When  $t$  is very small, dividing by  $t$  amplifies small prediction errors and can lead to unstable or exploding updates. Therefore, the clipping was increased to  $\epsilon = 1e - 2$ , giving  $t \in [0.01, 0.99]$ . This resolved the instability and allowed all four combinations to work at  $D = 2$ , consistent with the expected behaviour.

## Experiment Setup

Each of the 4 combinations is trained across 3 datasets (swiss roll, 8-mode gaussians, concentric circles) and 3 dimensions ( $D=2, 8, 32$ ), totalling 36 experiments. All models use the same hyperparameters: 25,000 training steps, batch size 1024, learning rate  $1e-3$ , hidden dimension 256, timesteps clipping to  $[0.01, 0.99]$  and 50 Euler sampling steps. 2048 samples are generated per configuration and projected back to 2D for visualisation. Results are organised as one figure per dataset with 4 rows (one per combination) and 3 columns (one per dimension).

## Results

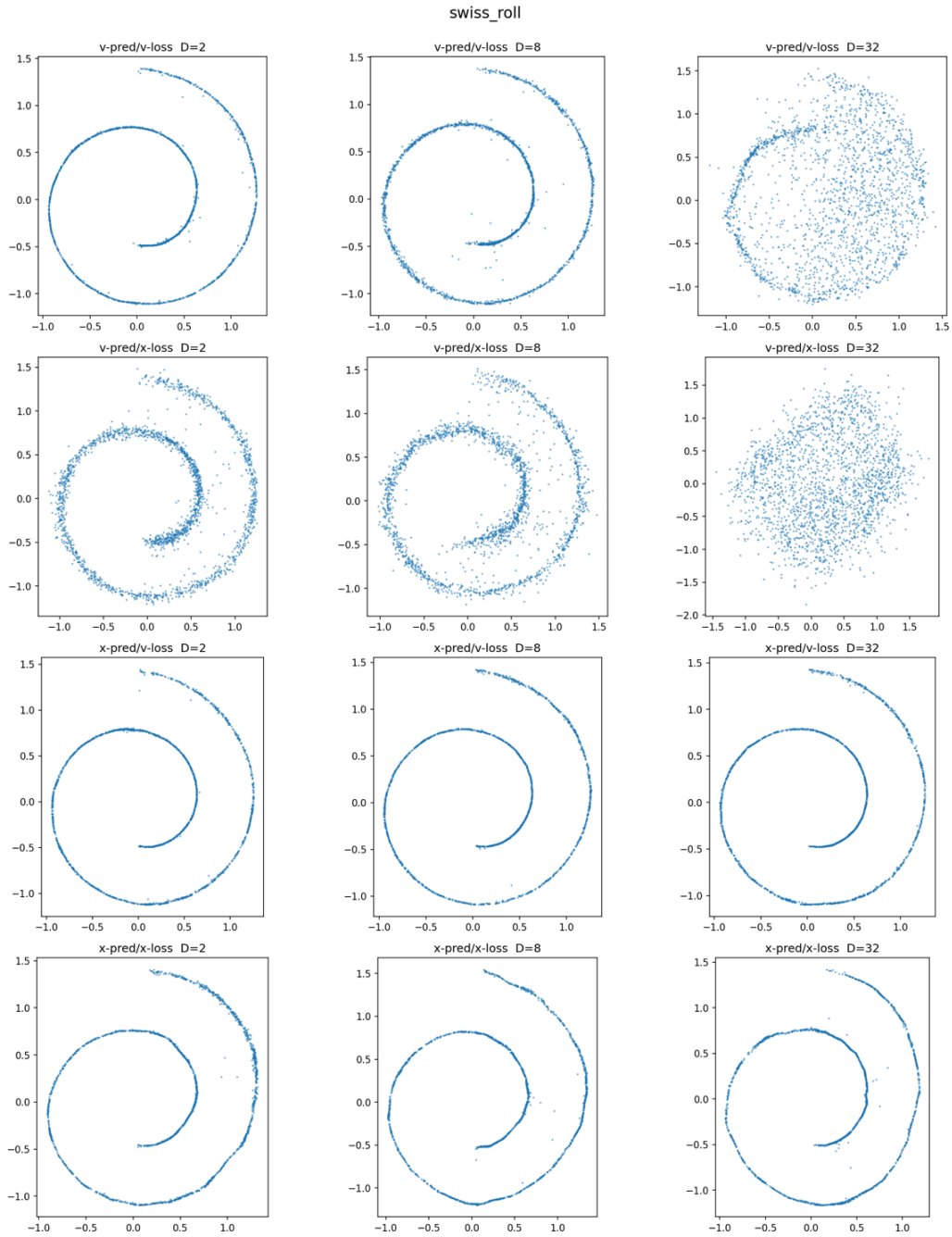


Figure 2.1: All 4 prediction/loss combinations on swiss roll at  $D=2, 8, 32$ .

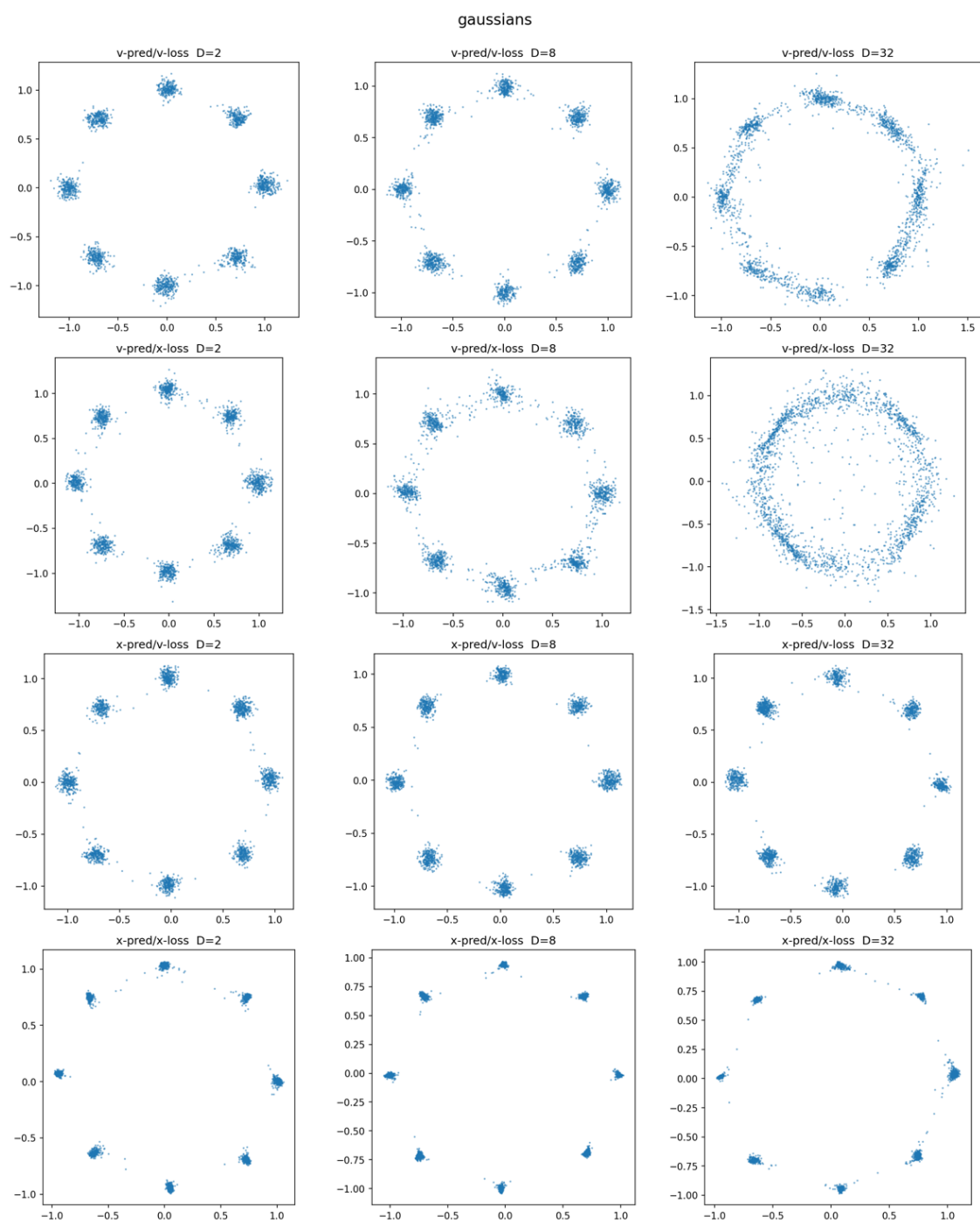


Figure 2.2: All 4 prediction/loss combinations on gaussians at  $D=2, 8, 32$ .

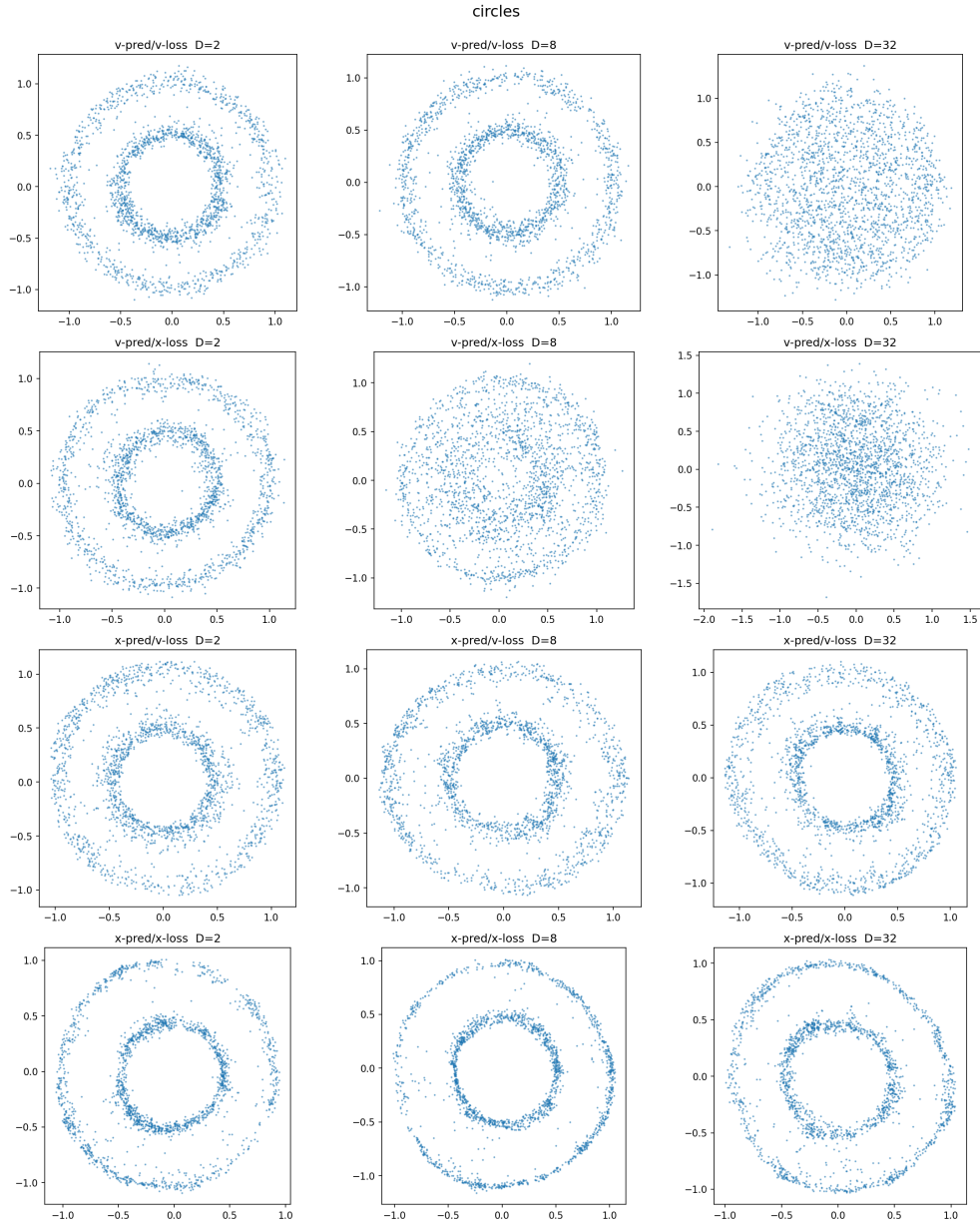


Figure 2.3: All 4 prediction/loss combinations on concentric circles at  $D=2, 8, 32$ .

At  $D=2$ , all four combinations produce recognisable samples across all three datasets. The spiral structure of swiss roll, the 8 clusters of gaussians, and the concentric rings of circles are all recovered.

At  $D=8$ , all four combinations still produce reasonable samples. V-pred/v-loss and x-pred/v-loss maintain clear structure. V-pred/x-loss becomes slightly noisier. X-pred/x-loss continues to work well.

At  $D=32$ , v-pred/v-loss and v-pred/x-loss degrade significantly, producing diffuse blobs that lose fine structure (spiral arms, distinct modes, ring separation). X-prediction scales much better: x-pred/v-loss is the strongest combination, producing clear spiral structure on swiss roll, distinct 8 Gaussian modes, and well-separated concentric circles. X-pred/x-loss also works at  $D=32$  but is slightly noisier than x-pred/v-loss, with somewhat less defined structure.

## 2.3 Questions

### Question 1: Which prediction type scales to high dimensions?

Based on the results, **x-prediction** scales better to high ambient dimensions. Both x-pred/v-loss and x-pred/x-loss produce stable and visually accurate samples at  $D=2$ , 8, and 32 across all datasets. Among them, x-pred/v-loss performing best overall, preserving the main geometric structures most clearly as the dimension increases. In contrast, **v-prediction** works well only in low dimension. At  $D=2$ , both v-pred/v-loss and v-pred/x-loss generate reasonable samples. At  $D=8$ , v-pred/v-loss still maintains clear structure, while v-pred/x-loss becomes slightly noisier but remains recognisable. However, at  $D=32$ , both v-pred methods collapse. The generated samples become diffuse blobs and lose the important dataset structures (spiral arms, distinct modes, ring separation). Therefore, x-pred is more robust to increasing ambient dimension, while v-pred becomes unreliable in high-dimensional settings.

### Question 2: Does the loss space (x-loss, v-loss) affect which pred type success or failure?

The choice of loss space has a meaningful effect on sample quality, but it does not change the main success or failure pattern. Overall, v-loss outperforms x-loss for both prediction types. This is likely because Euler sampling relies on velocity at every step, using the update  $z = z - v \cdot dt$ , so training in v-space directly improves the quantity that controls generation. The effect is especially clear for **x-prediction**. Although the model outputs  $x_{\text{pred}}$ , sampling still requires converting it into velocity using  $v_{\text{pred}} = (z_t - x_{\text{pred}})/t$ . Therefore, training with v-loss directly penalises errors in the velocity used for sampling. As a result, x-pred/v-loss produces sharper and more structured samples than x-pred/x-loss at  $D=2$ , 8, and 32. For **v-prediction**, v-pred/v-loss also performs better than v-pred/x-loss at  $D=2$  and  $D=8$ , where the latter appears noticeably noisier. However, at  $D=32$ , both v-prediction methods fail and lose the underlying data structure, regardless of the loss type.

Overall, these results suggest that **prediction type** is the primary factor affecting generation quality, while the choice of loss space provides an additional improvement. In particular, v-loss is most beneficial when combined with x-prediction.

### Question 3: Why does x-prediction work at high D while v-prediction fails?

The main reason is the difference between the prediction targets. The data lies on a 2-dimensional manifold embedded in a  $D$ -dim space. In **x-prediction**, the model predicts the clean data point  $x$ . Since  $x$  lies on this low-dimensional manifold, the target remains effectively low-dimensional even when the ambient dimension  $D$  increases. Therefore, the model only needs to learn the underlying 2D data structure. In contrast, **v-prediction** requires the model to predict  $v = \epsilon - x$ . The noise term  $\epsilon$  is sampled from a full  $D$ -dimensional Gaussian distribution, so the velocity target is also full-dimensional. As  $D$  increases, most of the velocity target is dominated by random noise rather than useful data structure. This makes the learning problem much harder, especially for a fixed-capacity model.

This explains why x-prediction scales better to high dimensions. It keeps the prediction target low-rank and easier to learn. The x-pred/v-loss combination works particularly well because the model still predicts the low-dimensional clean data  $x$ , while the loss is computed in velocity space. This directly encourages accuracy in the velocity used during Euler sampling, leading to better generation quality.



### Part 3: Can we rescue v-prediction?

#### Implementation

Two approaches are tested to rescue v-prediction at  $D=32$  on the swiss roll dataset, motivated by Section 4 of the RAE paper:

1. **Model capacity** is increased by varying the hidden dimension across  $h=32, 128, 256, 384, 512$ , and  $768$  (default is  $256$ ). In high ambient dimensions, the velocity target  $v = \epsilon - x$  becomes more difficult to learn because it is full-dimensional. Therefore, a wider network may provide enough capacity to better approximate the velocity field.
2. An **RAE-style schedule shift** is applied to the timestep sampling distribution. The sampled timestep  $t$  is transformed as  $t' = \frac{\alpha \cdot t}{1 + (\alpha - 1)t}$ , where  $\alpha = \sqrt{\text{intrinsic dim} / \text{dim}} = \sqrt{2/32} = 0.25$ . This transformation shifts more training samples toward  $t = 0$ , where  $z_t$  is closer to the clean data and the velocity field contains stronger learnable structure. In contrast, at high  $t$ ,  $z_t$  is dominated by noise, making the velocity target harder to learn. Therefore, the schedule shift aims to focus training on the more informative low-noise region of the trajectory.

#### Experiment Setup

All experiments use v-prediction with v-loss on the swiss roll dataset at  $D=32$ . Each model is trained for 25,000 steps with batch size 1024 and learning rate  $1e-3$ . A total of 12 models are trained: 6 hidden widths  $\times$  2 conditions (no shift, with shift). Samples are generated using 50 Euler steps and then projected back to 2D. Ground truth is shown in Row 0 for reference.

#### Results

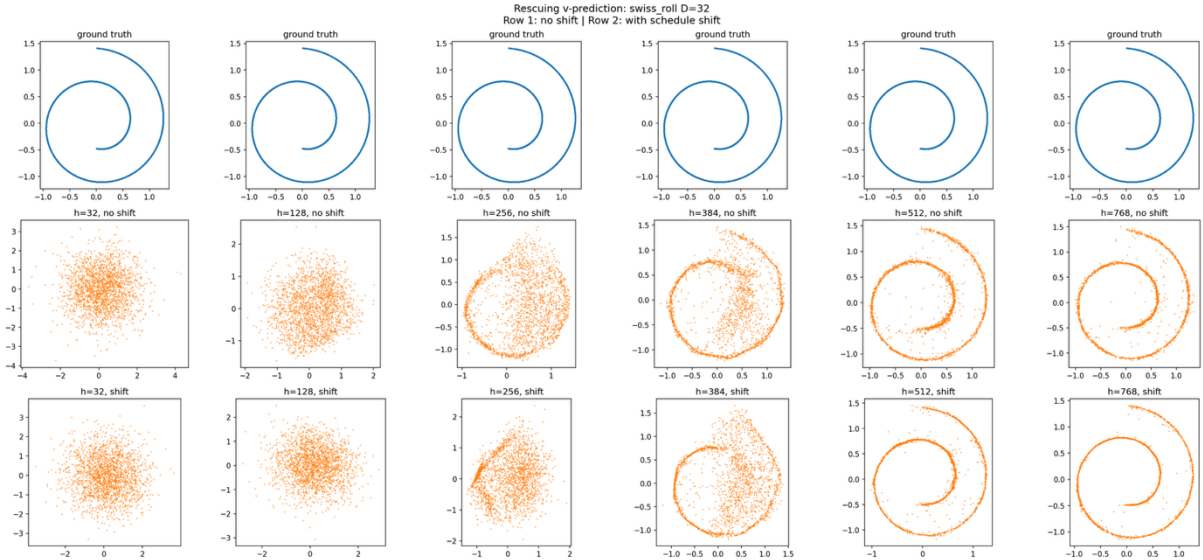


Figure 3.1: Rescuing v-prediction on the swiss roll at  $D=32$ . Row 0: ground truth. Row 1: no schedule shift. Row 2: with schedule shift. Columns:  $h=32, 128, 256, 384, 512, 768$ .

Without schedule shift,  $h=384, h=512$ , and  $h=768$  all recover a recognizable swiss roll spiral, demonstrating that model width alone is sufficient to rescue v-prediction at  $D=32$ . Sample



quality improves progressively with increasing width.  $h=384$  produces a noisy spiral, while  $h=512$  and  $h=768$  produce cleaner and more defined structures. This is because  $v$ -prediction targets  $v = \varepsilon - x$ , which inherits the full rank of the Gaussian noise across all 32 dimensions. Wider models have greater capacity to disentangle the weak signal of  $x$  from the dominant noise, allowing them to learn the swiss roll structure successfully.

In contrast,  $h=32$  and  $h=128$  fail completely under both conditions, producing randomly scattered points with no discernible structure. This confirms RAE Theorem 1: when model width is insufficient,  $v$ -prediction cannot converge regardless of schedule shift, as the model lacks the capacity to represent the full-rank  $v$  target.  $h=256$  without shift, the baseline from Part 2, also fails, though a faint spiral shape begins to emerge, suggesting the model sits on the boundary of having enough capacity but cannot fully recover the structure without additional support.

Adding schedule shift further reduces noise and scatter for  $h=512$  and  $h=768$ , producing the cleanest results overall. The shift compresses  $t$  toward smaller values using  $\alpha = 0.25$ , reducing the effective noise level during training so the model sees more signal from  $x$ . However, schedule shift consistently hurts performance at borderline widths.  $h=256$  with shift unexpectedly collapses into a diffuse blob, performing worse than its no-shift counterpart. Similarly,  $h=384$  with shift produces a more blurred spiral than  $h=384$  without shift. This is because  $\alpha = 0.25$  compresses  $t$  too aggressively toward small values, leaving the model undertrained at high noise levels. Since generation must start from pure noise at  $t=1$  and denoise through all timesteps, the model fails to handle high noise levels during generation, degrading the output quality at intermediate widths where capacity is already limited.

### 3.1 Question:

#### 1. Is $v$ -prediction's failure fundamental or can it be overcome?

The failure of  $v$ -prediction at  $D = 32$  is not fundamental. The results show that **increasing the model** width alone can recover clear swiss roll samples, even without applying schedule shift. This suggests that the poor performance observed in Part 2 was mainly caused by insufficient model capacity. In high dimensions, the velocity target  $v = \varepsilon - x$  is full-rank and harder to learn, so the default model was not wide enough to represent it accurately. **Schedule shift** provides an additional improvement when the model width is large enough. However, it can be less helpful or even counterproductive at borderline widths, where the model may not have enough capacity to benefit from the shifted timestep. Overall, these results indicate that  $v$ -prediction can work at  $D = 32$ , but it requires sufficient model capacity. Therefore, the earlier failure should be understood as a capacity limitation rather than an inherent weakness of  $v$ -prediction itself.

#### 2. What approach(es) did you try? Compare the compute cost.

Two approaches motivated by RAE Section 4 are tested. The first approach is **model width scaling**.  $V$ -prediction is trained using six hidden dimensions:  $h = 32, 128, 256, 384, 512$ , and  $768$ . The default  $x$ -prediction setting uses  $h = 256$  and already works well at  $D = 32$ . In contrast,  $v$ -prediction requires  $h = 512$  to achieve similar sample quality. This means the  $v$ -prediction model needs to be roughly  $2\times$  wider. Since each hidden linear layer maps from width  $h$  to width  $h$ , the number of hidden-layer parameters scales approximately as  $h^2$ .

Therefore, increasing the width from 256 to 512 doubles  $h$  but increases the hidden-layer parameter count by about  $2^2 = 4$  times. This also increases the compute cost per training step. The second approach is a **dimension-dependent schedule shift**, using  $\alpha = \sqrt{(2/32)} = 0.25$ . This has almost no extra compute cost, since it only applies a simple transformation to the sampled timestep. For wider models, such as  $h = 512$  and  $h = 768$ , the schedule shift provides a small additional improvement in sample quality. However, for borderline widths, such as  $h = 256$  and  $h = 384$ , it hurts performance. Therefore, schedule shift is not consistently beneficial across all model sizes.

Overall, the results show that v-prediction can match x-prediction quality at  $D = 32$ , but only with a wider model. Compared with the default x-prediction model, v-prediction needs approximately  $2\times$  larger hidden width, which leads to about  $4\times$  more hidden-layer parameters and higher training cost. The schedule shift gives only a minor additional benefit and does not replace the need for increased model capacity.

### 3. Compare how x-prediction and v-prediction respond to your changes.

X-prediction and v-prediction do not respond to the changes in the same way. For **x-prediction**, the model predicts the clean data point  $x$ . Since the data lies on a 2D manifold, the target remains effectively low-dimensional even when the ambient dimension  $D$  increases. Therefore, the default hidden width is already enough, and increasing model width or applying schedule shift does not provide much extra benefit. For **v-prediction**, the model predicts  $v = \varepsilon - x$ . Since  $\varepsilon$  is sampled from a full  $D$ -dimensional Gaussian distribution, the velocity target also becomes full-dimensional. As  $D$  increases from 2 to 32, the target becomes much harder to learn while the model capacity stays fixed. This explains why v-prediction fails with the default width but improves when the hidden width is increased from 256 to 512. Schedule shift can help v-prediction when the model is wide enough because it focuses training on lower-noise timesteps where the data structure is clearer. However, at borderline widths, it can hurt because the model receives less training in the high-noise region, which is still needed for generation from noise.

Therefore, the two parameterisations do not behave the same way: x-prediction is already robust at high  $D$ , while v-prediction requires more capacity as  $D$  increases.

### 4. v-prediction is used successfully in real image generation systems. Why might the situation be different for those models compared to these toy datasets?

Real image generation is different from these toy datasets in several important ways, so the rank mismatch problem is less severe in practice.

First, real image generation models usually do not work directly in raw pixel space. Instead, they often operate in a learned latent space produced by an autoencoder or VAE. This latent space keeps important semantic image information while removing unnecessary pixel-level variation. In contrast, these toy experiments operate directly in the ambient data space. The data has intrinsic dimension 2 but is embedded in  $D = 32$ , so the target  $v = \varepsilon - x$  is dominated by 32-dimensional noise and contains only a small 2D data signal. In a VAE latent space, the data signal is spread more meaningfully across the latent dimensions, making v-prediction easier to learn.

Second, real generative models have much larger capacity. In these experiments, v-prediction at  $D = 32$  only works well after increasing the hidden width from 256 to 512. Real image generation models use very large transformer architectures with far more parameters, so they have enough capacity to learn complex high-dimensional velocity targets.

Third, natural images contain strong spatial structure and redundancy. Modern architectures, such as transformers with attention, can exploit correlations between image regions. This makes the learning problem easier. In contrast, the toy datasets are randomly projected into high-dimensional space, which spreads the 2D structure across all dimensions and removes useful spatial patterns. This makes the problem artificially difficult for a fixed-size MLP.

Finally, real systems use carefully designed noise schedules. These schedules are tuned to balance learning across different noise levels. In this assignment, a simple fixed schedule shift with  $\alpha = 0.25$  was tested. It helped wide models but hurt borderline-width models because it reduced training in the high-noise region. Real systems use more carefully balanced schedules, which can reduce this trade-off.

Overall, v-prediction works better in real image generation because the latent representation is more meaningful, the models are much larger, the data has exploitable structure, and the noise schedule is carefully designed.

## Part 4: One-Step Generation

### 4.1 Sampling Efficiency

#### Implementation

The best model from Part 2 (x-pred/v-loss) is used for all three datasets across  $D=2, 8$ , and  $32$ . Each model is trained with the same hyperparameters as Part 2 (25,000 steps, batch size 1024, lr  $1e-3$ , hidden dimension 256, timesteps clipping to  $[0.01, 0.99]$ ). After training, 2048 samples are generated using Euler ODE sampling at 8 different step counts: 1, 2, 5, 10, 20, 50, 100, and 200. For  $D > 2$ , samples are projected back to 2D for visualisation. Results are organised as one figure per dataset with 3 rows ( $D=2, 8, 32$ ) and 9 columns (ground truth + 8 step counts)

#### Results

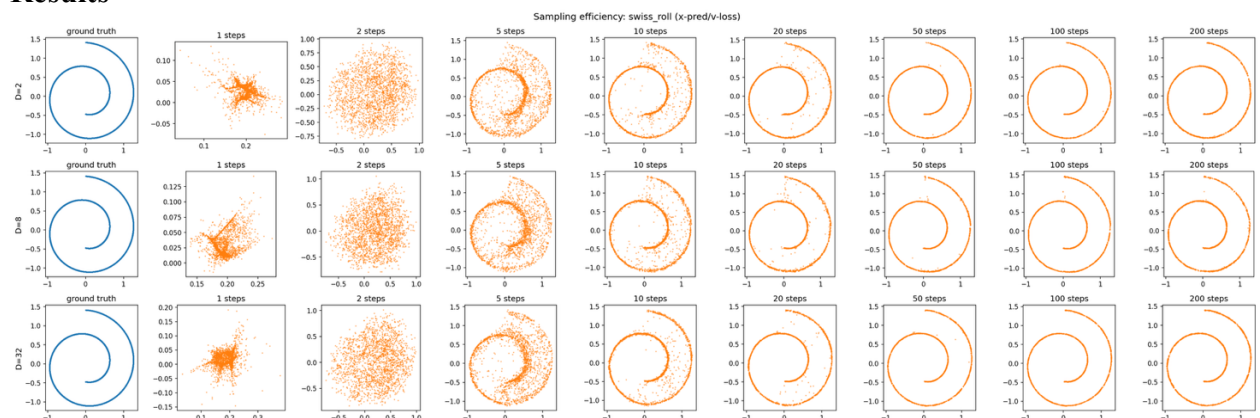


Figure 4.1.1: Sampling efficiency on swiss rolls (x-pred / v-loss) across  $D = 2, 8, 32$  and step count = 1, 2, 5, 10, 20, 50, 100, 200

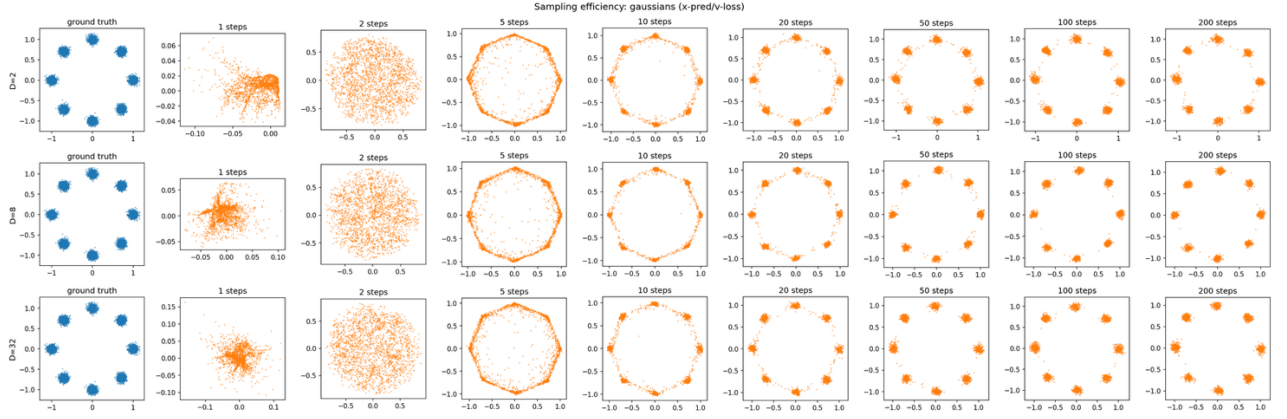


Figure 4.1.2: Sampling efficiency on 8-mode gaussians ( $x$ -pred /  $v$ -loss) across  $D = 2, 8, 32$  and step count = 1, 2, 5, 10, 20, 50, 100, 200

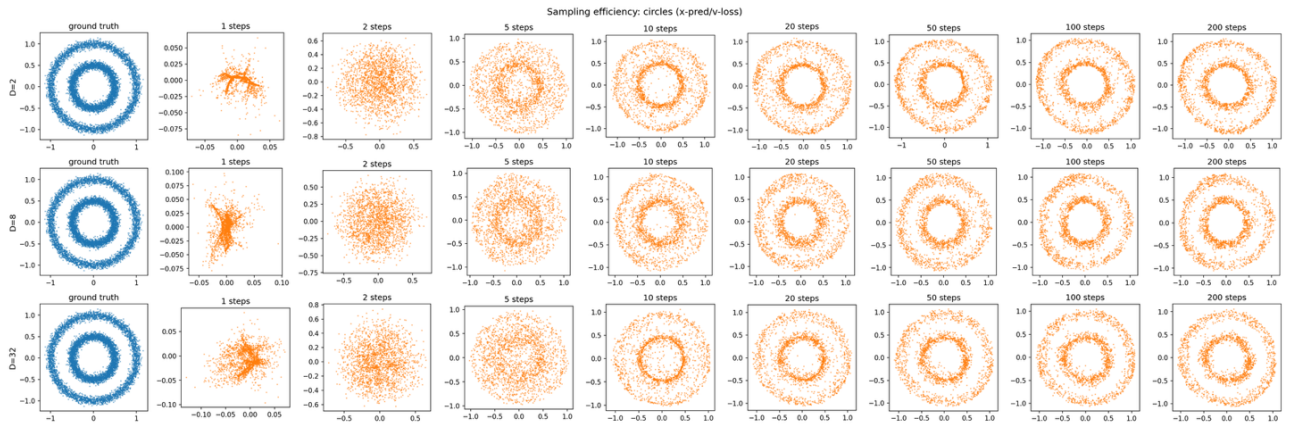


Figure 4.1.3: Sampling efficiency on concentric circles ( $x$ -pred /  $v$ -loss) across  $D = 2, 8, 32$  and step count = 1, 2, 5, 10, 20, 50, 100, 200

At high step counts, such as 50, 100, and 200, all three datasets generate clean samples that closely match the ground-truth distributions. The sample quality is still acceptable at 20 and 10 steps, although small degradation appears around the edges of the distributions.

A clear transition occurs at 5 steps. At this point, the overall structure of each dataset is still recognisable, but fine details are lost and the samples become more scattered. At 2 steps, the structure largely breaks down. At 1 step, generation fails completely, with samples collapsing into a small dense cluster near the origin. This happens because a single Euler step is too coarse to accurately approximate the continuous ODE trajectory from noise to data.

Importantly, the degradation pattern is similar across  $D = 2, 8$ , and 32. The same critical step counts appear for all ambient dimensions. This suggests that the required number of sampling steps depends mainly on the complexity of the data distribution, rather than the ambient dimension. This is consistent with the earlier result that  $x$ -prediction remains robust as  $D$  increases.

## 4.2 Mean Flow

### Implementation

The MeanFlow MLP extends the base FlowMatching MLP by accepting an additional horizon input  $h = t - r$ , where  $t$  is the current timestep and  $r$  is the target timestep. Both  $t$  and  $h$  are encoded as 128-dimensional sinusoidal embeddings and concatenated with the noisy input  $z_t$ , giving an input dimension of `data_dim` + 256. The network uses 5 hidden layers of width 256 with ReLU activations and a final linear layer outputting `data_dim` values.

The model predicts the mean clean data  $X = f_\theta(z_t, t, h)$ , from which the mean velocity is derived as  $U = (z_t - X)/t$ . The training objective follows from the definition of mean velocity over the interval  $[r, t]$ :

$$u_\theta(z_t, r, t) = \frac{1}{t-r} \int_r^t v_\theta(z_\tau, \tau) d\tau.$$

Multiplying both sides by  $(t-r)$  and differentiating with respect to  $t$  gives:

$$v_\theta(z_t, t) = u_\theta(z_t, r, t) + (t-r) \frac{d}{dt} u_\theta(z_t, r, t)$$

Rearranging with  $h = t - r$ :

$$u = v - h \cdot \frac{du}{dt}$$

Training uses `fm_ratio` to split between two branches:

With probability `fm_ratio`, we set  $h = 0$  and train with standard x-prediction/v-loss. The model predicts  $X = f_\theta(z_t, t, 0)$ , converts to velocity  $\hat{v} = (z_t - X)/t$ , and minimises  $|\hat{v} - v_{\text{target}}|^2$  where  $v_{\text{target}} = \epsilon - x$ .

With probability  $(1 - \text{fm\_ratio})$ , we sample  $h > 0$  and apply the MeanFlow consistency loss:

1. Sample  $t$  and  $r$  independently from  $U(0.01, 0.99)$ , ensure  $r < t$ , and set  $h = t - r$ .
2. Compute the mean velocity  $U = (z_t - f_\theta(z_t, t, h))/t$ . Its total derivative  $\frac{dU}{dt}$  is computed using a single JVP call. The JVP tangent is set to  $(v_{\text{target}}, 1, 1)$  for the inputs  $(z_t, t, h)$ . This is because, when  $r$  is fixed and  $t$  changes,  $z_t$  moves along the velocity direction  $v_{\text{target}}$ ,  $t$  changes at rate 1, and  $h = t - r$  also changes at rate 1. Therefore, the JVP computes the total derivative along the flow trajectory rather than only the partial derivative with respect to  $h$ .
3. Assemble the target from the rearranged identity:  $u_{\text{tgt}} = v_{\text{target}} - h \cdot \frac{dU}{dt}$ , with stop-gradient applied to the target.
4. Minimise  $|U - u_{\text{tgt}}|^2$ .

Sampling:

Given a trained MeanFlow model, samples are generated by Euler integration from  $t = 1$  (noise) to  $t = 0$  (data). At each step, the model predicts  $X = f_\theta(z, t, h)$  where  $h = 1/K$  for  $K$  total steps, the velocity is recovered as  $V = (z - X)/t$ , and the sample is updated as  $z \leftarrow z - V \cdot h$ . For one-step generation ( $K = 1$ ), this simplifies to  $z_0 = f_\theta(z_1, 1, 1)$ : the model directly maps pure noise to a clean sample in a single forward pass.

## Experiment Setup

The models are trained for 25,000 steps on all three datasets: swiss roll, gaussians, and concentric circles. All experiments use  $D = 32$ , batch size 1024, learning rate  $10^{-3}$ , and hidden dimension 256. Timesteps are clipped to  $[0.01, 0.99]$  for numerical stability. Both  $t$  and  $r$  are sampled independently from  $U(0.01, 0.99)$ . The MeanFlow model uses the x-prediction/v-loss parameterisation, which was the best-performing configuration in Part 2.

The main MeanFlow setting uses a flow matching ratio of 0.75. This means that 75% of training steps use  $h = 0$  with the standard x-pred/v-loss objective, while the remaining 25% use  $h > 0$  with the MeanFlow consistency loss. To examine the effect of this ratio, an additional experiment with flow matching ratio 0.5 is also tested, but it produced slightly less sharp samples. Therefore, 0.75 was selected because the stronger  $h = 0$  anchor provides a more reliable velocity estimate for the consistency loss.

For comparison, a standard Flow Matching model is trained under the same conditions using x-pred/v-loss. Samples from both models are generated using 1, 2, and 5 sampling steps, then projected back to 2D for visualisation.

## Results

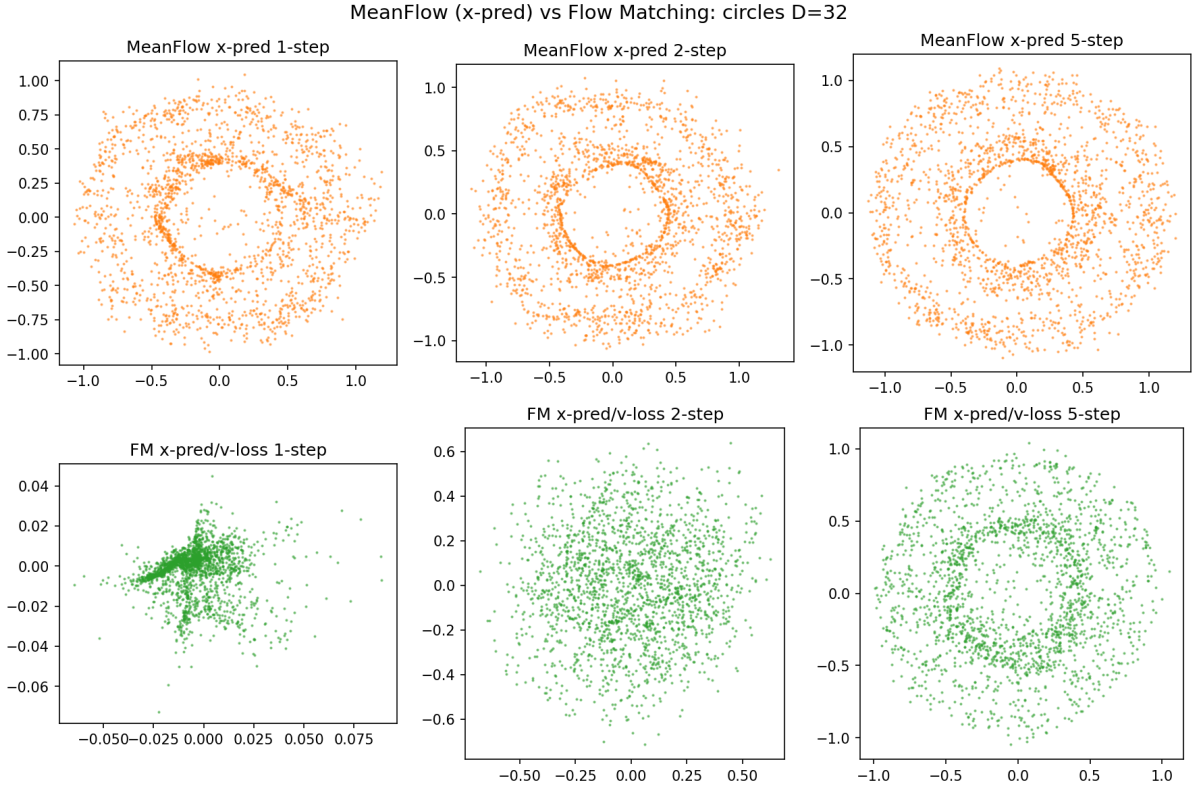


Figure 4.2.1: MeanFlow vs flow matching (x-pred/v-loss) on concentric circles at  $D=32$ , with 1, 2, and 5 sampling steps.



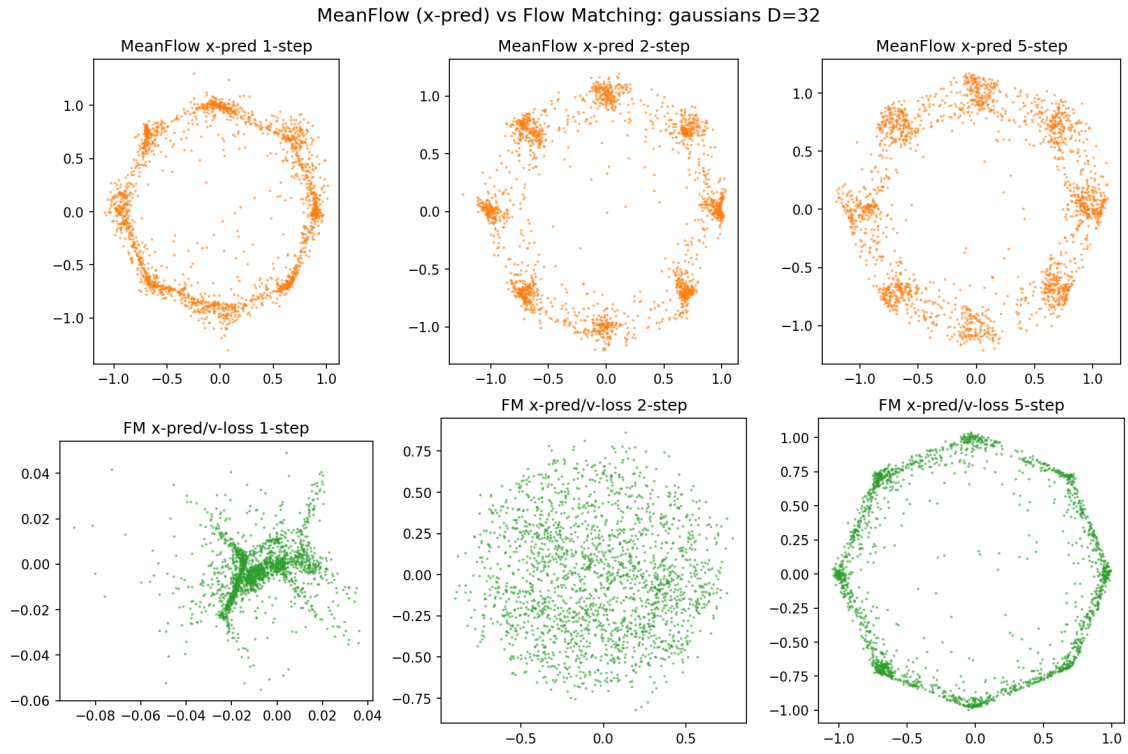


Figure 4.2.2: MeanFlow vs flow matching (x-pred/v-loss) on 8-Gaussians at  $D=32$ , with 1, 2, and 5 sampling steps.

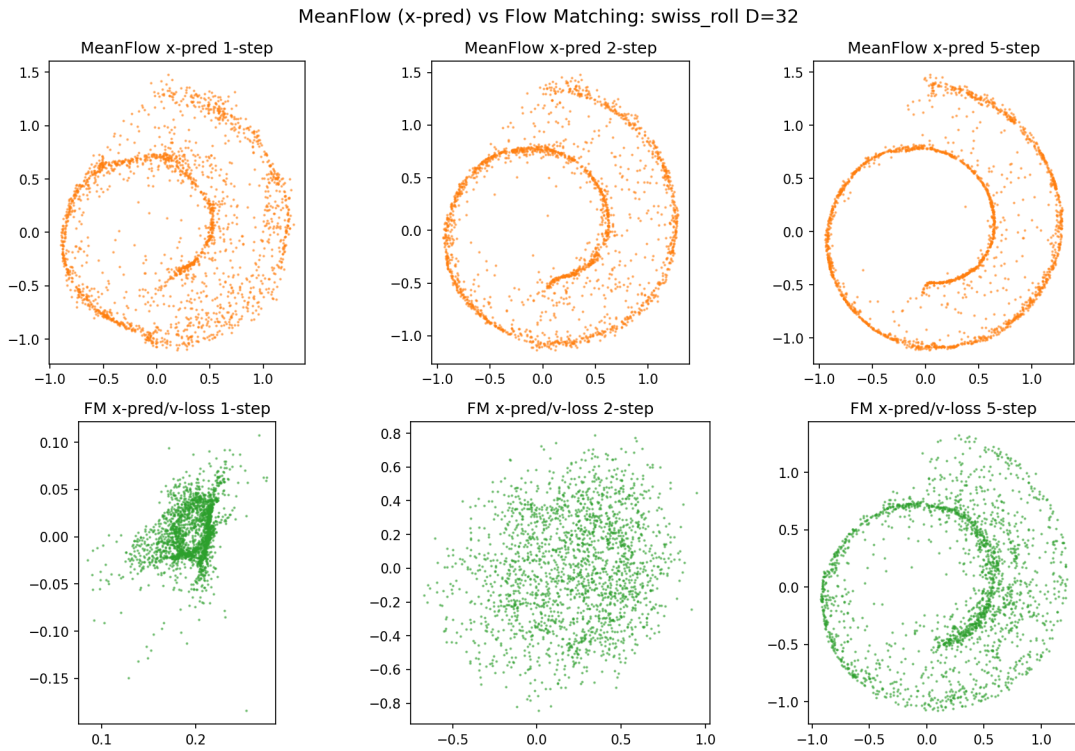


Figure 4.2.3: MeanFlow vs flow matching (x-pred/v-loss) on swiss roll at  $D=32$ , with 1, 2, and 5 sampling steps.



Across all three datasets at  $D=32$ , MeanFlow with x-prediction consistently outperforms standard Flow Matching at low step counts.

For swiss roll, MeanFlow at 1 step captures the correct spatial extent, though the boundaries are diffuse. At 2 steps the spiral becomes clearer, and at 5 steps the arms are well defined. Standard FM at 1 step collapses near the origin, begins spreading at 2 steps, and only recovers a recognisable spiral at 5 steps.

For concentric circles, MeanFlow at 1 step preserves the general concentric circular structure but the two rings are not clearly separated. At 2 steps the inner ring becomes visible, and at 5 steps both rings are well resolved with some noise. Standard FM at 1 step collapses to a small cluster, shows partial circular structure at 2 steps, and recovers two rings at 5 steps but with noticeably more noise than MeanFlow.

For 8-mode gaussians, MeanFlow at 1 step produces a roughly continuous ring rather than eight distinct clusters. At 2 steps the clusters begin to separate, and at 5 steps the eight modes are clearly visible but still with noise. Standard FM at 1 step collapses near the origin, shows some spread at 2 steps, and displays an octagonal cluster arrangement at 5 steps.

## Questions

### 1. Why did you choose this prediction type for MeanFlow?

**X-prediction** is chosen because x-pred/v-loss was the best-performing configuration in Part 2, especially at  $D = 32$  where v-prediction degraded. As discussed in Part 2 Q3, the x-prediction target is the clean data  $x$ , which lies on a low-rank 2-dimensional manifold regardless of the ambient dimension  $D$ . This keeps the learning problem easier and more stable in high-dimensional settings without requiring a wider model.

Therefore, the MeanFlow model uses the same x-pred/v-loss parameterisation to keep the high-dimensional robustness observed in Part 2 while still training the model to produce useful velocity information for sampling.

### 2. Describe the core idea behind MeanFlow.

MeanFlow extends standard Flow Matching by learning an average transition over a time interval, rather than only learning the instantaneous velocity at a single timestep. Standard Flow Matching learns the local velocity field  $v(z_t, t)$ , which only describes how to move from the current noisy point for a small step. Therefore, sampling usually requires many Euler steps from  $t = 1$  to  $t = 0$ .

MeanFlow introduces a horizon  $h = t - r$ , where  $r$  is the target earlier timestep. Instead of predicting only the local velocity at  $t$ , the model learns the mean movement over the interval from  $t$  to  $r$ . In the x-prediction formulation, the model outputs  $X(z_t, t, h)$ , which represents the clean-data-style prediction associated with that horizon. This enables one-step generation because the horizon can cover the full trajectory. Starting from pure noise at  $t = 1$  with  $h = 1$ , the model can directly predict the clean sample at  $t = 0$ :  $z_0 = \text{model}(z_1, 1, 1)$ .

Thus, MeanFlow learns to “look ahead” across the trajectory, allowing generation with fewer steps than standard Flow Matching.

### 3. Why is the $h = 0$ portion needed?

The  $h = 0$  portion is needed to anchor MeanFlow to the true Flow Matching objective. When  $h = 0$ , the model is trained using standard x-pred/v-loss, so its prediction is directly supervised by the ground-truth velocity  $v_{target} = \epsilon - x$ . This gives the model a reliable local velocity estimate.

Although the  $h > 0$  branch also uses the ground-truth velocity as the JVP trajectory direction, the target  $u_{tgt} = v_{target} - h \cdot \frac{dU}{dt}$  contains the model's own total derivative  $\frac{dU}{dt}$  with stop-gradient. Early in training, this derivative is unreliable, making  $u_{tgt}$  a noisy target. The  $h = 0$  branch provides stable, direct supervision that helps the model learn accurate local predictions first, which in turn makes  $\frac{dU}{dt}$  more meaningful and allows the  $h > 0$  consistency loss to effectively teach the model over longer horizons.

### 4. Compare the training cost of MeanFlow to standard flow matching.

MeanFlow is harder to train than standard Flow Matching because the  $h > 0$  consistency step requires extra computation. A standard Flow Matching step only needs one model forward pass to predict  $x$  and compute the MSE loss.

For MeanFlow, the  $h > 0$  step must compute both the mean velocity  $U$  and the total derivative  $\frac{dU}{dt}$ . This is done using a JVP call, which propagates both the normal model output and derivative information through the network. Therefore, the JVP is roughly comparable to two forward passes, so one  $h > 0$  MeanFlow step costs approximately twice as much as one standard Flow Matching step.

Since the final setting uses `fm_ratio = 0.75`, 75% of the steps are standard  $h = 0$  steps and 25% are  $h > 0$  consistency steps. The average cost per training step is therefore approximately  $0.75 \times 1 + 0.25 \times 2 = 1.25$ . So, with the same 25,000 training steps, MeanFlow costs about 1.25 times more to train than standard Flow Matching. However, this extra training cost can be offset during sampling, because MeanFlow can generate useful samples in very few steps, even one step, while standard FM requires many Euler steps.

### 5. Compare the MeanFlow-generated samples against the GT across all three datasets.

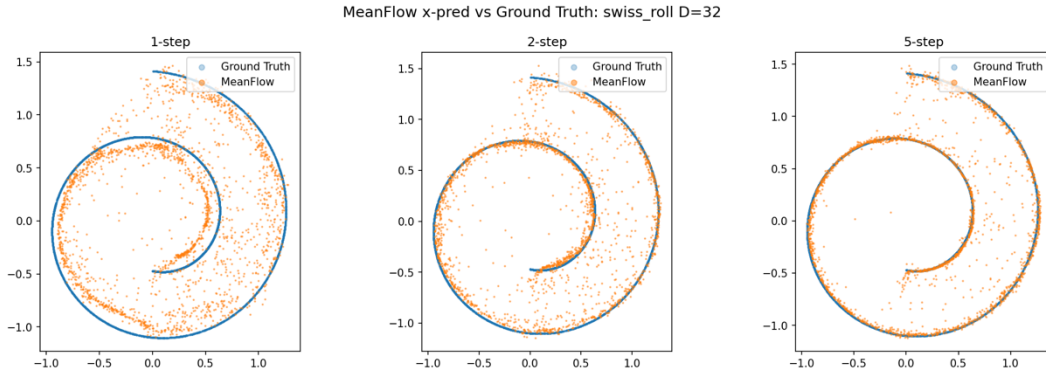


Figure 4.3.1: MeanFlow x-prediction samples (orange) overlaid with ground truth (blue) on swiss roll at  $D=32$

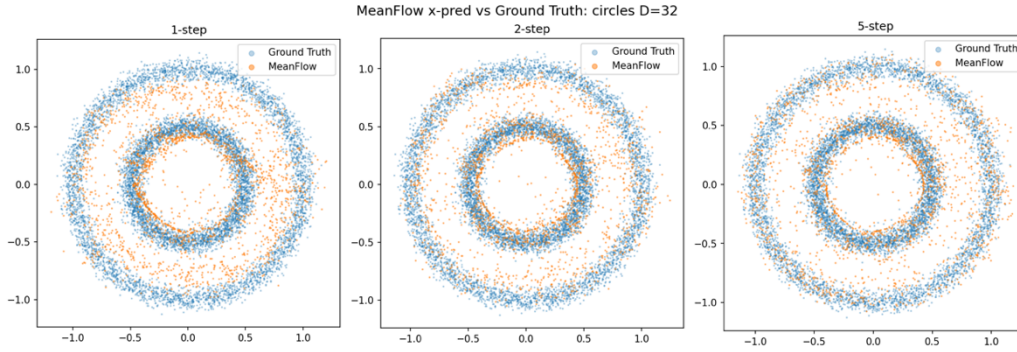


Figure 4.3.2: MeanFlow  $x$ -prediction samples (orange) overlaid with ground truth (blue) on concentric circles at  $D=32$

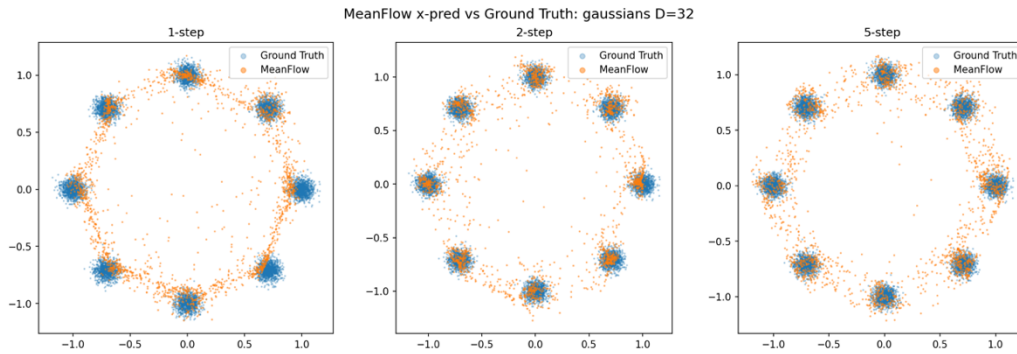


Figure 4.3.3: MeanFlow  $x$ -prediction samples (orange) overlaid with ground truth (blue) on 8-mode gaussians at  $D=32$

Comparing MeanFlow samples against the ground truth using the overlay figures, the swiss roll and circles datasets show smooth quality degradation as the step count decreases. At 5 steps, both closely match the ground truth, although some noise remains. At fewer steps, the samples become more diffuse, with the spiral arms filling into an annulus for swiss roll and the two rings blending together for circles. However, no major structural artifacts appear. This is expected because both are continuous distributions, so averaging over nearby trajectories mainly causes blurring rather than introducing spurious structure.

The gaussians dataset shows a qualitatively different artifact. The ground truth consists of 8 sharply separated clusters, but MeanFlow at 1 step produces a roughly uniform ring, with points scattered between modes rather than concentrated on them. This is a mode-averaging effect caused by the mean  $x$ -prediction formulation. When the horizon is large, such as  $h=1$  for one-step generation, a noise sample can be compatible with multiple Gaussian modes. The mean prediction averages over trajectories targeting different modes, placing the prediction between modes rather than committing to one. Since the 8 modes are arranged in a ring, these averaged predictions fill in the gaps, producing a continuous ring. As the step count increases, this artifact diminishes. At 5 steps, each step uses a shorter horizon,  $h=0.2$ , and by later steps  $z$  has moved closer to a single mode. The short-range prediction then no longer spans multiple modes, allowing the clusters to become more distinct.

Overall, MeanFlow matches the ground truth increasingly well as the number of sampling steps increases, but one-step generation can produce averaging artifacts on multi-modal data, especially for the gaussians dataset.